

CRBL: Reference Manual

Taine J. Rossini

Version 1.0.0.

Abstract

This document lists every setting recognised by `crbl`, the chemically reacting boundary-layer solver, along with the arguments of its two solver classes, `BoundaryLayer` and `ViscousInteraction`. Anything not listed here is ignored.

Command line

```
crbl [option]
```

```
--help      Displays the help message and exits.
--bl=<file>  Runs a boundary-layer simulation using a .py configuration file.
--vi=<file>  Runs a viscous-interaction simulation over a flat plate or
             wedge using a .py configuration file.
```

One option is given per run.

- `--bl=<file>` runs the boundary-layer described by the configuration file.
- `--vi=<file>` runs the viscous interaction described by the same kind of file.
- Running with no argument, or with `--help`, prints the banner and exits.

The two options select which solver is run, not which one the file builds; a file that builds a `ViscousInteraction` but is passed to `--bl` does nothing.

The configuration script

The configuration file is a Python script, executed inside `crbl`'s own namespace. It does not need to import anything itself: every name below is already available in that namespace when the script runs.

Name	Description
<code>config</code>	The global solver configuration object, described below.
<code>BoundaryLayer</code>	The boundary-layer solver class.
<code>ViscousInteraction</code>	The viscous-interaction solver class.
<code>np</code>	<code>numpy</code> .
<code>eqc</code>	The <code>equilibrium-c</code> module, for building an equilibrium inflow.
<code>GasModel</code> , <code>GasState</code> , <code>GasFlow</code>	The <code>gdtk</code> gas objects.

A script has three jobs, in this order.

- Set the `config` fields it wants to change from their defaults.

- Call `config.init_gas_model()`, which loads the gas-model and, if `chem_model_file` is set, the reactor. This must happen before the solver object is constructed, since the constructor reads `config.gmodel`.
- Construct exactly one `BoundaryLayer` or `ViscousInteraction` object.

Solver settings

All of these are attributes of the global `config` object.

Required settings

Key	Type	Description
<code>gas_model_file</code>	string	Path to the gas-model file. The run aborts if it is empty or the file is missing.

`config.init_gas_model()` must then be called to load it.

Models and physics

Key	Type	Default	Description
<code>chem_model_file</code>	string	""	Path to the reaction file. Required when <code>reacting</code> is on; ignored when it is off.
<code>reacting</code>	bool	False	Solves a species conservation equation for each species with a finite-rate source term. With this off the composition is frozen at the edge value.
<code>axisymmetric</code>	bool	False	Solves the axisymmetric form, scaling the streamwise coordinate by the body radius <code>r_arr</code> .
<code>adiabatic</code>	bool	False	Zero enthalpy gradient at the wall instead of a fixed <code>T_wall</code> .
<code>catalytic</code>	bool	False	Fully catalytic wall. Also enables the diffusive heat flux, which is zero otherwise.

Wall models

- **Non-catalytic**, the default, fixes a zero species gradient across the wall.
- **catalytic = True** holds the wall composition at equilibrium for the local edge pressure and wall temperature, and is the only setting under which the diffusive heat flux q_d is computed (zero otherwise). It has no effect unless `reacting` is also on.
- **adiabatic = True** replaces the isothermal wall with a zero-gradient condition on the total enthalpy, so `T_wall` is ignored. It is not implemented for reacting flows.

Grid

Key	Type	Default	Description
<code>eta_end</code>	float	6.0	End of the similarity domain, where the freestream begins. Must sit outside the layer.
<code>n_y_cells</code>	int	100	Number of points across the layer, evenly spaced in η .
<code>n_x_cells</code>	int	100	Number of streamwise marching stations.

`n_x_cells` is used differently by the two solvers.

- **BoundaryLayer** overwrites it with `len(x_arr)` on construction, so the value set in the script only matters as a way of sizing the edge arrays that are then passed in.
- **ViscousInteraction** builds its own station distribution from `x_start` to `x_end`, so the setting takes effect directly.

Iteration

Each station is solved using Picard iteration: momentum, then the stream function, then energy, then species, repeated until the largest change in any profile falls below `tol`.

Key	Type	Default	Description
<code>max_iters</code>	int	100	Maximum Picard iterations per station. The station is accepted as it stands when this is reached.
<code>tol</code>	float	1e-4	Convergence tolerance on the largest absolute change in U , g or any species profile between iterations.
<code>omega</code>	float	0.1	Under-relaxation factor. Each new profile enters as $\omega \phi_{new} + (1 - \omega) \phi_{old}$, so small values are slow but stable.

Output

Key	Type	Default	Description
<code>verbose</code>	bool	True	Prints a line per station with the wall clock, iteration count and residual, and a line per viscous-interaction iteration.
<code>export</code>	bool	True	Writes <code>loads.dat</code> and <code>output.vts</code> at the end of the run.

BoundaryLayer settings

Solves the flow-field over an arbitrary geometry, given the inviscid edge condition as a function of the body arc length x . Every array is indexed by station and must be `n_x_cells` long, that being the length of `x_arr`.

Argument	Type	Default	Description
<code>x_arr</code>	array (nx,)	required	Body arc length, m. Must be increasing and must start above zero.
<code>u_e_arr</code>	array (nx,)	required	Edge velocity, m/s.
<code>T_e_arr</code>	array (nx,)	required	Edge temperature, K.
<code>P_e_arr</code>	array (nx,)	required	Edge pressure, Pa. The streamwise pressure gradient is calculated from this array directly.
<code>massf_e_arr</code>	array (nx, nsp)	required	Edge mass fractions, one row per station, ordered as in the gas-model. Must be a numpy array.
<code>r_arr</code>	array (nx,)	None	Body radius, m. Only used when <code>axisymmetric</code> is on, and required by it.
<code>T_wall</code>	float	300.0	Isothermal wall temperature, K. Ignored when <code>adiabatic</code> is on.

The first station must sit at $x > 0$; the transformed streamwise coordinate is zero at the leading edge and the solution is undefined there.

ViscousInteraction settings

Solves the viscous-inviscid interaction over a flat plate or wedge. The boundary-layer is solved, its displacement thickness then fed to the tangent-wedge method to get a new edge pressure distribution, and then iterated on until the pressure converges.

Argument	Type	Default	Description
<code>u</code>	float	required	Freestream velocity, m/s.
<code>p</code>	float	required	Freestream pressure, Pa.
<code>T</code>	float	required	Freestream temperature, K.
<code>massf</code>	list (<code>nsp</code> ,)	required	Freestream mass fractions, ordered as in the gas-model.
<code>T_wall</code>	float	300.0	Isothermal wall temperature, K.
<code>x_start</code>	float	1e-5	First station, m. Must be larger than zero.
<code>x_end</code>	float	0.1	Last station, m.
<code>theta</code>	float	0.0	Wedge half-angle, <i>degrees</i> . Zero is a flat plate.
<code>tol</code>	float	1.0	Convergence tolerance on the edge pressure, Pa.
<code>d_star_idx</code>	int	100	Number of points used to fit the displacement-thickness curve.

A few things are worth knowing about the loop.

- A non-zero `theta` is resolved with an oblique shock first; the post-shock state replaces the inflow and the stations run along the wedge surface.
- `tol` is an absolute change in Pa between iterations, not a normalised residual, so set it against the pressure level of the case.
- The loop is capped at 30 iterations, which is not adjustable.
- `d_star_idx` thins the displacement-thickness curve before interpolating, keeping numerical wobble out of the surface slope. Too many points and the slope oscillates, taking the shock-angle relation outside its valid range and crashing the run.

Output

With `export` on, two files are written into the working directory at the end of the run, both overwritten without warning.

`loads.dat`

One row per station, comma separated, with the header

```
x (m), d (m), d* (m), p (Pa), tau (Pa), q_c (W/m^2), q_d (W/m^2)
```

Being the arc length, boundary-layer height, displacement thickness, edge pressure, wall shear stress, and the convective and diffusive heat fluxes.

`output.vts`

The full field as a VTK structured grid, on the physical (x, y) coordinates. The point data carries `vel.x`, `T`, `rho`, `k`, `mu` and one `massf-<species>` array per species.

A complete configuration script

A reacting boundary-layer, run with `--bl`.

```
# Setup solver.
config.gas_model_file = "air-7sp.gas"
config.chem_model_file = "air-7sp.chem"
config.reactiving = True
config.n_x_cells = 500
config.n_y_cells = 500
config.init_gas_model()

# Freestream conditions.
u1 = 8601
p1 = 530
T1 = 1936
massf1 = [0.767, 0.233, 0, 0, 0, 0, 0]

# Edge conditions along the body.
x = np.linspace(0.0001, 0.13, config.n_x_cells)
P_e = p1 * (1 + 0.05 / np.sqrt(x))
u_e = np.full(config.n_x_cells, u1)
T_e = np.full(config.n_x_cells, T1)
mass_f_e = np.full((config.n_x_cells, config.gmodel.n_species), massf1)

BoundaryLayer(x_arr=x, u_e_arr=u_e, T_e_arr=T_e, P_e_arr=P_e, massf_e_arr=mass_f_e)
```

Invoked as,

```
crbl --bl=job.py
```

The viscous interaction over the same plate, run with `--vi`. The inflow is brought to equilibrium first, which is the usual way to start these.

```
# Setup solver.
config.gas_model_file = "air-7sp.gas"
config.chem_model_file = "air-7sp.chem"
config.reactiving = True
config.n_x_cells = 200
config.n_y_cells = 100
config.export = False
config.catalytic = True
config.omega = 0.5
config.init_gas_model()

# Freestream conditions.
u1 = 8601
p1 = 530
T1 = 1936
massf1 = [0.767, 0.233, 0, 0, 0, 0, 0]

# Convert to an equilibrium inflow.
eq = eqc.EqCalculator(config.gmodel.species_names)
Y0 = np.array(massf1)
X0 = eq.YtoX(Y0)
X1 = eq.pt(p=p1, T=T1, Xs0=X0)
Y1 = eq.XtoY(X1).tolist()

ViscousInteraction(u=u1, p=p1, T=T1, massf=Y1, x_end=0.13)
```

Invoked as,

```
crbl --vi=job.py
```

Using CRBL as a library

The same two classes can be driven from an ordinary Python script, which is the way to get at the profiles rather than just the wall loads. The settings are identical; the only differences are that the script imports what it needs, and that you must call `run()` itself and unpack the result.

```
# Load libraries.
from crbl import BoundaryLayer, config
import matplotlib.pyplot as plt
import numpy as np

# Setup solver.
config.gas_model_file = "air-7sp.gas"
config.chem_model_file = "air-7sp.chem"
config.reactiving = True
config.init_gas_model()

# Edge conditions.
x = np.linspace(0.0001, 0.13, config.n_x_cells)
P_e = 530 * (1 + 0.05 / np.sqrt(x))
u_e = np.full(config.n_x_cells, 8601)
T_e = np.full(config.n_x_cells, 1936)
mass_f_e = np.full((config.n_x_cells, config.gmodel.n_species), [0.767, 0.233, 0, 0, 0, 0, 0])

# Call solver.
bl = BoundaryLayer(x_arr=x, u_e_arr=u_e, T_e_arr=T_e, P_e_arr=P_e, massf_e_arr=mass_f_e)

eta, y, bl_height, delta_star, U, massf, rho, T, k, mu, tau_w, q_c, q_d = bl.run()

# Plot the displacement thickness.
plt.plot(x, delta_star)
plt.show()
```

What `BoundaryLayer.run()` returns

Thirteen arrays, in this order. `nx` is the number of stations, `ny` is `n_y_cells` and `nsp` is the number of species.

Name	Shape	Description
<code>eta</code>	<code>(nx, ny)</code>	Similarity coordinate, the same row repeated.
<code>y</code>	<code>(nx, ny)</code>	Physical wall-normal coordinate, m.
<code>bl_height</code>	<code>(nx,)</code>	Height at which u first reaches 99% of the edge value, m. NaN where the layer does not get there.
<code>delta_star</code>	<code>(nx,)</code>	Displacement thickness, m.
<code>u</code>	<code>(nx, ny)</code>	Velocity, m/s. Dimensional, not the normalised profile.
<code>massf</code>	<code>(nx, ny, nsp)</code>	Mass fractions.
<code>rho</code>	<code>(nx, ny)</code>	Density, kg/m ³ .
<code>T</code>	<code>(nx, ny)</code>	Temperature, K.
<code>k</code>	<code>(nx, ny)</code>	Thermal conductivity, W/m/K.

Name	Shape	Description
<code>mu</code>	<code>(nx, ny)</code>	Dynamic viscosity, Pa s.
<code>tau_w</code>	<code>(nx,)</code>	Wall shear stress, Pa.
<code>q_c</code>	<code>(nx,)</code>	Conductive heat flux at the wall, W/m ² .
<code>q_d</code>	<code>(nx,)</code>	Diffusive heat flux at the wall, W/m ² .

What `ViscousInteraction.run()` returns

`ViscousInteraction` is driven the same way: construct it, then call `run()` directly. It returns six arrays, all `(nx,)`, taken from the last completed iteration.

Name	Description
<code>x</code>	Station coordinate along the surface, m.
<code>d_star</code>	Displacement thickness, m.
<code>P_e</code>	Edge pressure, Pa.
<code>tau_w</code>	Wall shear stress, Pa.
<code>q_c</code>	Conductive heat flux at the wall, W/m ² .
<code>q_d</code>	Diffusive heat flux at the wall, W/m ² .

Improving stability

Settings worth adjusting first when a run will not converge.

- Lower `omega`, e.g. to 0.05-0.1, and raise `max_iters` to match.
- Tighten `tol` to 1e-5 or below on reacting runs; the chemistry is not resolved at the default.
- Increase `n_y_cells` to better resolve the wall gradients that set the heat flux and shear stress.
- Check `eta_end` is far enough out that the profiles have flattened onto the edge condition before the domain ends.
- Increase `n_x_cells`, or start the first station closer to the leading edge, where the streamwise steps are largest in transformed space.
- Adjust `d_star_idx` when a viscous-interaction run reports that the tangent-wedge method failed.